

Hardware Dataflow for Convolutional Neural Network Accelerator

Adiwena Putra

University Center of Excellence
on Microelectronics
Institut Teknologi Bandung
Bandung, Indonesia

Trio Adiono

University Center of Excellence
on Microelectronics
Institut Teknologi Bandung
Bandung, Indonesia

Nana Sutisna

University Center of Excellence
on Microelectronics
Institut Teknologi Bandung
Bandung, Indonesia

Infal Syafalni

University Center of Excellence
on Microelectronics
Institut Teknologi Bandung
Bandung, Indonesia

Rahmat Mulyawan

University Center of Excellence
on Microelectronics
Institut Teknologi Bandung
Bandung, Indonesia

Abstract—Convolutional Neural Network (CNN) is one the most used Artificial Neural Network in AI system. While this type of network managed to achieve high accuracy for many tasks, it requires high computational power which can't be handled by the general purpose CPU. GPU usually becomes the first choice to accelerate the CNN, but unfortunately, GPU has low energy efficiency which made it not suitable for edge computing application. To address this problem, current research focuses on ASIC and FPGA implementation of hardware accelerator which has better energy efficiency. In this paper, we propose hardware dataflow for CNN accelerator. This dataflow is scalable for multicore system and also can be implemented on ASIC or FPGA. We implemented the proposed dataflow on FPGA. The proposed dataflow managed to achieve 30 GOPS throughput in FPGA. We also provide the analytical framework for this dataflow which can be used to estimated energy consumption and data reusability under various computation workloads.

Index Terms—dataflow, hardware accelerator, convolutional neural network

I. INTRODUCTION

Convolutional layer is the basic building block for CNN. The area application of CNN is usually to be found on digital image processing. Before the rise of deep learning, people tend to use the hand-designed kernel for feature extraction using the convolution operation. In the year of 2012, Alex et al. [1] win the ImageNet competition using AlexNet which contain 5 convolutional layer and 3 fully connected layer. The result of this competition mark the downfall of hand-designed kernel and the beginning of deep learning era. In the year of 2014, Karen et al. [2] wins the ImageNet competition using VGG-16 which contain 16 layer. The trend seems to shown that the deeper the layer is, the better the accuracy. However, it is not easy to train the deep neural network because of the vanishing gradient problem. Just when we thought it can't be going any deeper again, in the year of 2016, Kaiming et al. [3] found ResNet which utilize the skip connection. By using skip connection, the vanishing gradient problem can be solved and we can train even more deeper neural network (≥ 56 layers).

The increasing depth of neural network proposes new challenge to the world of computing. With very deep neural network, the number of MAC operations can easily surpasses 500M. The architecture of general purpose CPU is not suitable for this computation. The easy solution of this problem is by using GPU, with already existing framework to design the program that can utilize GPU, GPU becomes the first choice to accelerate the neural network.

As time goes by, the era of IoT started to kick in, and the edge computing application demands also rise. People started to notice that GPU is not suitable for edge computing application because of its form factor and low energy efficiency. To address this problem, the current research starts to develop hardware accelerator in FPGA and ASIC because of its energy efficiency [4]. FPGA, in particular, becomes an interesting solution because of its flexibility to reconfigure the logic fabric.

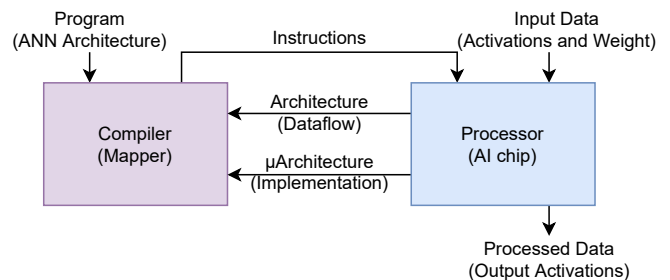


Fig. 1. The operation analogy of AI chip and processor. Figure adopted from [5]

The research of AI accelerator, both in FPGA or ASIC, falls into the new category of AI chip research. It turns out that designing AI chip and general purpose processor has similar design components. Figure 1 shows the operation analogy between AI chip and general purpose processor. In general purpose processor, we have compiler to translate the program

into the instructions that can be understood by the processor. Similarly, in AI chip, we have the mapper to translate the ANN architecture (number of layer, number of hidden units, activation function to use, etc.) into the instructions for AI chip. In processor, the compiler needs the information of architecture (instruction set architecture) and microarchitecture to be able to generate the right instruction set for the targeted processor. The architecture in processor is the abstract model of the computer that dictate the flow of data in processor, while microarchitecture is the realization of architecture. The similar analogy of architecture in processor is known as hardware dataflow in AI chip.

Similar to the architecture in processor which is limited in number (E.g., x86, ARM, MIPS, etc.), there are only limited number of dataflow in AI chip. During the time of this paper being made, there 4 dataflow for AI chip:

- **Weight Stationary Dataflow** [6]–[8]: optimize the weight reuse.
- **Output Stationary Dataflow** [9], [10]: optimize the output activation reuse.
- **Input Stationary Dataflow** [11]: optimize the input activation reuse.
- **Row Stationary Dataflow** [12], [13]: optimize the overall reuse.

Each of these dataflows has pros and cons. Depending on the workload, one dataflow can be more efficient than the others. Weight, output, and input stationary dataflow optimize the reuse of respective data type, while row stationary dataflow try to optimize overall data with the trade off complex NoC design.

In this paper, we present the novel dataflow based on input stationary dataflow. The dataflow use three memory level to fully optimize the reuse of input activations. The dataflow is also based on multicore architecture, made it scalable to be implemented on edge computing device with limited resources. The main contribution of this paper is as following.

- We propose the novel input stationary dataflow for CNN accelerator based on systolic core for scalable design.
- We perform the detail profiling of the dataflow using the loopnest.
- We provide the analysis framework of dataflow which can be used to estimated the energy consumption and to measure the data re-usability.

The rest of the paper is structured as follows. Section II provides the background knowledge that is needed to understand the computation in the dataflow. Section III describe the dataflow and hardware architecture for CNN accelerator. Section IV provides the analysis framework to estimate the energy and to measure the data re-usability. The framework also provides insides into how to optimize the architecture based on the workload. Section V provides the result of demonstration of the dataflow in FPGA. Finally, section VI conclude the result of this works.

II. CNN BACKGROUND

A. Convolution Operation

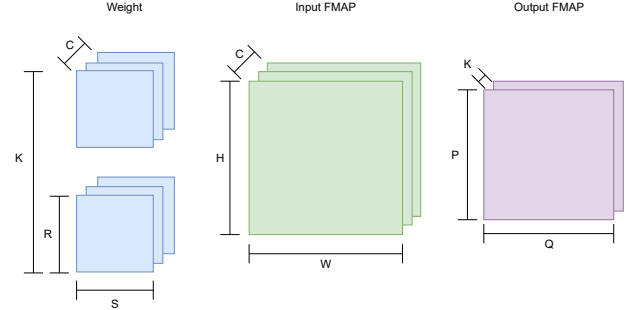


Fig. 2. Convolution operation and its parameters

In CNN, The convolution operation accounts for 90% of operations and dominates the runtime [14]. Figure 2 shows the illustration of 2D convolution in CNN. The C channel input activation, known as FMAP, is convolved with K sets of 3D filter, resulting in K channel output FMAP. The detail of convolution operation can be describe in following equation.

$$O[k][p][q] = \sum_{c=0}^{C-1} \sum_{r=0}^{R-1} \sum_{s=0}^{S-1} I[c][Up+r][Uq+s] \times F[k][c][r][s] + B[k] \quad (1)$$

$$0 \leq k < K, 0 \leq c < C, 0 \leq r < R, 0 \leq s < S$$

$$P = \frac{H - R + 2\text{pad}}{U} + 1, Q = \frac{W - S + 2\text{pad}}{U} + 1$$

where:

- I = Input FMAP
- F = Filter weight
- B = Bias
- O = Output FMAP
- H = Input FMAP height
- W = Input FMAP width
- C = Input channel
- R = Filter kernel height
- S = Filter kernel width
- K = Output Channel
- P = Output FMAP height
- Q = Output FMAP width
- pad = Number of padding
- U = Number of stride in convolution

B. Convolution as Matrix Multiplication

As can be seen form Eq.1, the convolution is performed as tensor multiplication between 3D FMAP tensor and 4D filter tensor over the iteration variable c , r , and s . We want to simplify this multiplication into 2D tensor, matrix matrix multiplication. In this way, we can compute the multiplication using the systolic array architecture which is more easy and scalable to be implemented on various devices.

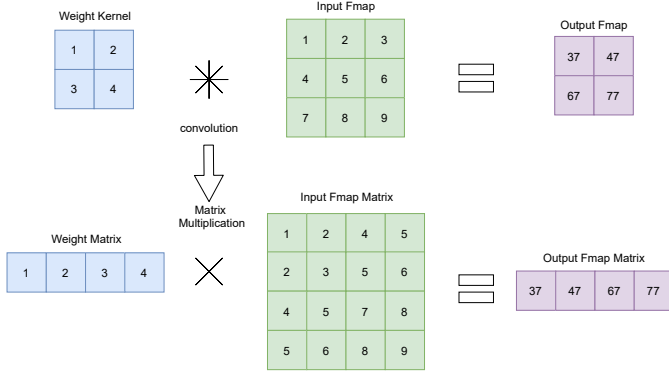


Fig. 3. Convolution as matrix multiplication

Figure 3 shows the illustration of how the convolution can be computed as matrix matrix multiplication. To convert 4D filter tensor into 2D matrix, we just need to merge the rank in 4D tensor. The filter as 4D tensor has the dimensionality of $F[K][C][R][S]$. In 2D matrix representation, the dimensionality becomes $F[K][CRS]$. To convert the 3D FMAP tensor into 2D matrix, we need to unroll every computation patch in column fashion. Formally, the way to convert 3D FMAP tensor into 2D matrix is by using *im2col()* function [15]. Algorithm 1 describe the *im2col()* function. The algorithm receive 3D tensor FMAP $fmap[C][H][W]$ and converts it into 2D matrix $mat_fmap[CRS][PQ]$.

Algorithm 1 Im2col Function

```

1: /* Variables: */
2: fmap[C][H][W]
3: mat_fmap[CRS][PQ]
4:
5: for c=0 in range(0,C) do
6:   for p=0 in range(0,P) do
7:     for q=0 in range(0,Q) do
8:       for r=0 in range(0,R) do
9:         for s=0 in range(0,S) do
10:          mat_fmap[c*R*S+r*S+s][p*Q+q] =
            fmap[c][U*p+r][U*q+s]
11:        end for
12:      end for
13:    end for
14:  end for
15: end for

```

III. HARDWARE DATAFLOW & MICROARCHITECTURE

In this section, we will describe the dataflow using loopnest pseudocode. We also provides the example of how we can implement this dataflow in hardware. In our dataflow, we separate the memory into 3 memory levels: DRAM memory, L1 memory, and L0 memory. In ASIC implementation, L1 and L0 memory represent the cache memory. While in FPGA implementation, L1 memory can be thought as BRAM memory and L0 memory as LUT register. The memory levels also

vary in term of storage capacity and energy cost. The closer the memory to the computing unit, the smaller the storage capacity it has and the more energy cost efficient it is.

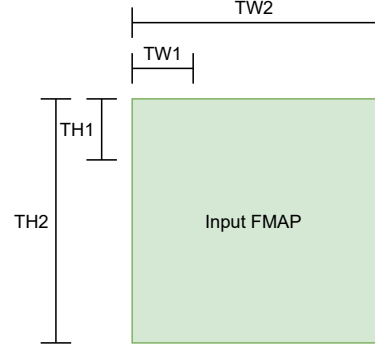


Fig. 4. Tiling strategy for our proposed dataflow

TABLE I
TILING PARAMETER DESCRIPTION

Dataflow Parameter	Description	Value
TW1	Systolic array width	Determined
TW2	Row batch number	W/TW1
TH1	Number of FMAP row to be processed	size(L1)/(WxC)
TH2	Column batch number	H/TH1

One of the challenge in designing AI accelerator is the mapping process of CNN architecture to hardware dataflow. To be able to map any size of convolution to our dataflow, we performs specific tiling strategy on the input FMAP. Figure 4 shows the tiling parameter for our dataflow and Table I shows the descriptions of tiling parameter. *TW1* value is determined from the systolic array width which can be vary depending on the available resource. For the optimum size of systolic array, choose *TW1* to be the greatest common divisor across all FMAP size in CNN, and select the systolic array width to be the maximum filter size in CNN (*RS*).

Our proposed dataflow is based on input stationary dataflow. In this dataflow, we optimize the reuse of input activations. In fact, we managed to design our dataflow so all input activations only need to be read once from DDR memory. Algorithm 2 shows our propose dataflow for CNN accelerator. Here, variables *ifmap*, *ofmap*, and *weight* act as DDR memory representation, while variables *ifmap_buffer*, *weight_buffer*, and *ofmap_buffer* act as L1 memory representation. Variable *ifmap_pe* acts as L0 memory and it is reserved for storing input activations only.

There are 8 loops level in our dataflow. The outermost loop in line 10 describe the loop iteration over *TH2* parameter. In this loop, the read operation is performed for input activations (line 11). During one read process, the accelerator will read the first *TH1* rows of FMAP in every channel and store it in L1 memory (variable *ifmap_buffer*). The second loop in line 12 describe the loop iteration over *C* parameter. In this loop, the accelerator will read weight value (line 13). In one read

Algorithm 2 Input stationary dataflow loop nest

```

1: /* Memory Level: */
2: ifmap[C][HW] // DDR Memory
3: ofmap[K][HW] // DDR Memory
4: weight[C][K][RS] // DDR Memory
5: ifmap_buffer[C][TH1*W] // L1 Memory
6: weight_buffer[K][RS] // L1 Memory
7: ofmap_buffer[K][TH1*W] // L1 Memory
8: ifmap_pe[RS][TW1] // L0 Memory
9:
10: for nh1 in range(0, TH2) do
11:   ifmap_buffer = LoadFmap(ifmap,C,TH1,W,nh1)
12:   for c in range(0, C) do
13:     weight_buffer = LoadWeight(weight,K,RS,c)
14:     for h1 in range(0, TH1) do
15:       for nw1 in range(0, TW2) do
16:         ifmap_pe = Fill(ifmap_buffer,RS,TW1,nw1,h1,c)
17:         for parallel core in range(0, Ncore) do
18:           K1,K2 = AssignBound(K,core)
19:           for parallel rs in range(0,RS) do
20:             for parallel w1 in range(0,TW1) do
21:               for k in range(K1, K2) do
22:                 idx = AddrIdx(w1,nw1,h1,nh1)
23:                 ofmap_buffer[k][idx] +=
24:                   weight_buffer[k][rs] *
25:                   ifmap_pe[rs][w1]
26:               end for
27:             end for
28:           end for
29:         end for
30:       end for
31:     ofmap = WriteFmap(ofmap_buffer, K, W1, nh1)
32:   end for

```

operation, the number of KRS weight value is stored in L1 memory. This process will be repeated for every input channel. The next two loop (line 14 and line 15) iterate over $TH1$ and $TW2$ parameters. In this loop, we fill L0 memory with input activations from L1 memory. During this process, we also applying any *on the fly logic* to the input activations from L1 memory. The *on the fly logic* is represented as *Fill()* function (line 16). This function can represent any combination of *on the fly logic* including padding, data ordering, and *im2col()* function. The next three loops (line 17, line 19, and line 20) are "for parallel" loops which mean there will be the parallel architecture implementation for these loops in our accelerator. Loops in line 19 and 20 describe the size of our systolic array which is $RS \times TW1$, while the loops in line 17 describe the number of systolic core in accelerator. Each cores will work on different sets of output channel. Finally, the final partial sum value (output FMAP) will be written out to DDR memory in the end of first loop (line 31).

There are many ways to implement this dataflow. Figure 5

shows one example of the dataflow implementation using 3 core of systolic array. Here, on the fly logic represents any process of ordering the data and any transform function of the data (E.g., *im2col()*, zeropadding, pooling, etc.). Host CPU acts as the master of AI accelerator, Host CPU initiates the acceleration process by sending trigger signal to control unit of hardware accelerator.

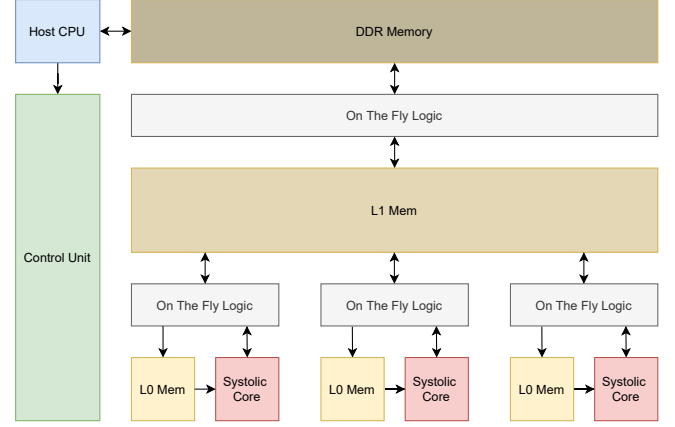


Fig. 5. Microarchitecture for the proposed dataflow

IV. DATAFLOW ANALYSIS FRAMEWORK

In this section, we provide the analysis framework for our proposed dataflow. The analysis framework can be used to estimate the energy consumption and data reusability for various workloads. This framework is derived from loopnest in Algorithm 2. There are 3 data types which will be our concern: IFMAP (input FMAP), OFMAP (output FMAP), and Weight. We neglect the bias data movement because the number of bias is small compare to other data types. In this framework, we will breakdown the number of memory access in each memory level for the 3 data types.

$$\begin{aligned}
 \text{DDR}_N &= \text{DDR}_I + \text{DDR}_W + \text{DDR}_O \\
 \text{L1}_N &= \text{L1}_I + \text{L1}_W + \text{L1}_O \\
 \text{L0}_N &= \text{L0}_I
 \end{aligned} \tag{2}$$

where:

DDR_N = Total DDR memory access
 L1_N = Total L1 memory access
 L0_N = Total L0 memory access
 DDR_I = IFMAP access on DDR memory
 L1_I = IFMAP access on L1 memory
 L0_I = IFMAP access on L0 memory
 DDR_W = Weight access on DDR memory
 L1_W = Weight access on L1 memory
 DDR_O = OFMAP access on DDR memory
 L1_O = OFMAP access on L1 memory

Start from DDR memory level, the value of DDR_I , DDR_W , and DDR_O can be derived from the number of *LoadFmap()*, *LoadWeight()*, and *WriteFmap()* function calls respectively in

Algorithm 2 (line 11, 13, and 31) multiplied by the number of byte data accessed in one function calls.

$$\begin{aligned} \text{DDR}_I &= \text{TH2} \times \text{DRbyte}_I \\ \text{DDR}_W &= \text{TH2} \times C \times \text{DRbyte}_W \\ \text{DDR}_O &= \text{TH2} \times \text{DRbyte}_O \end{aligned} \quad (3)$$

Where DRbyte is the number of byte accessed in one function call for respective data types. The value of DRbyte differ across batch layers because the dimension of FMAP and filter also vary. The exact value of these variables depends on the dataflow implementation.

While in L1 memory level, the value of $L1_I$ can be derived from the number of *Fill()* function calls in Algorithm 2 (line 16), this function will take a set of value form L1 memory, expand the value according to the padding and *im2col()* function applied in each respective layer and store the value in L0 memory. The value of $L1_W$ and $L1_O$ can be derived from the number of L1 memory access during the computing process (line 23).

$$\begin{aligned} L1_I &= \text{TH2} \times C \times \text{TH1} \times \text{TW2} \times L1\text{byte}_I \\ L1_W &= \text{TH2} \times C \times \text{TH1} \times \text{TW2} \times K \times L1\text{byte}_W \\ L1_O &= \text{TH2} \times C \times \text{TH1} \times \text{TW2} \times K \times L1\text{byte}_O \end{aligned} \quad (4)$$

Where L1byte is the number of byte accessed in one function call or the number of data read from L1 memory level for respective data type. For $L1\text{byte}_I$ the value will be different depending on the use of *im2col()* function to expand the IFMAP tile (in some cases, the filter size can be 1 by 1 dimension so *im2col()* function isn't needed). The value of L1byte also depends on microarchitecture implementation.

Finally, the value of $L0_I$ can be derived the same way as $L1_W$ and $L1_O$.

$$L0_I = \text{TH2} \times C \times H1 \times \text{TW2} \times K \times L0\text{byte}_I \quad (5)$$

Where $L0\text{byte}_I$ value also depends on microarchitecture implementation.

V. THE RESULT OF DEMONSTRATION

We implement the proposed dataflow in Xilinx FPGA Ultra 96. We run the YOLO v3 CNN [17] in our accelerator with the model as shown in Table III. For the implementation, we used 1 core and 8 bit data precision. Our systolic array size in 9 by 14 with $TW1 = 14$ and $RS = 9$. Tabel II shows the result of our dataflow implementation.

TABLE II
IMPLEMENTATION RESULTS

Clk Freq (Mhz)	250
BRAM	248
DSP	242
LUT	27.3k
FF	38.5k
GOPS	30
Power (Watt)	4.259

TABLE III
YOLO v3 MODEL

Layer Batch	Layer	Total Parameters
1	Zero Pad	432.00
	Conv, Leaky-ReLU, BN	
	Max Pooling (stride=2)	
2	Zero Pad	4,608.00
	Conv, Leaky-ReLU, BN	
	Max Pooling (stride=2)	
3	Zero Pad	18,432.00
	Conv, Leaky-ReLU, BN	
	Max Pooling (stride=2)	
4	Zero Pad	73,728.00
	Conv, Leaky-ReLU, BN	
	Max Pooling (stride=2)	
5	Zero Pad	294,912.00
	Conv, Leaky-ReLU, BN	
	Max Pooling (stride=2)	
6	Zero Pad	1,179,648.00
	Conv, Leaky-ReLU, BN	
	Max Pooling (stride=1)	
7	Zero Pad	4,718,592.00
	Conv, Leaky-ReLU, BN	
	Conv, Leaky-ReLU, BN	
8	Conv, Leaky-ReLU, BN	262,144.00
9	Conv, Leaky-ReLU, BN	32,768.00
10	Up Sampling	884,736.00
	Concat	
	Zero Pad	
	Conv, Leaky-ReLU, BN	
11	Conv, Leaky-ReLU, BN	4,608.00
12	Zero Pad	1,179,648.00
	Conv, Leaky-ReLU, BN	
13	Conv, Leaky-ReLU, BN	9,216.00

Running this workload using the analysis framework from section IV, we got the result in Figure 6. This Figure shows the memory access size (read and write) in each memory level for all data types (weight, IFMAP, OFMAP). As can be seen, our dataflow managed to minimize the DDR memory level access for all batch layers. This result also gives insight about the data reusability in our proposed dataflow. For example, in layer 3, memory access size from DDR is 0.7 MB, this data is stored in L1 memory, while the memory access size from L1 memory is 43.5 MB, this means the reusability of data is about $43.5/0.7 = 61.41$ times per data.

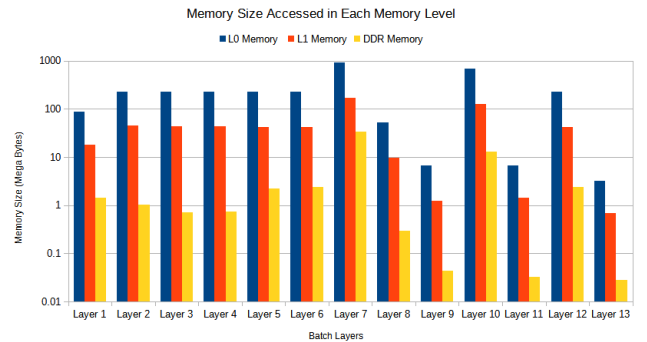


Fig. 6. Memory access size for YOLO v3 model

To get insight about the energy cost from data movement across memory level, we use the estimated normalize energy

cost from [16] as shown in Figure 7 and we get the estimated energy consumption as shown in Figure 8. This figure shown the percentage of energy consumption across batch layers. To get this results, we simply multiply the result from Figure 6 with the respective energy cost from Figure 7. As can be seen, our dataflow can push energy consumption from DDR memory as low as 24% (layer 3 and 4), but in some layers the dominated energy consumption is still from DDR memory access (at worst 76% in layer 7). This data give us insight which layers can be optimize more, in particular one way to optimize this result is by modifying the workload and tiling strategy for YOLO model.

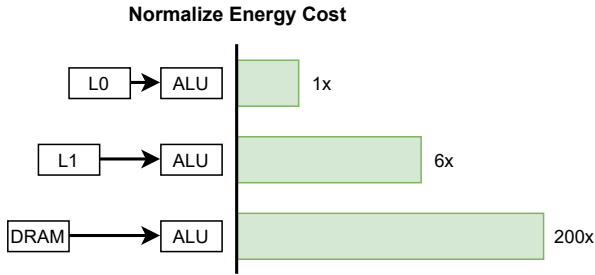


Fig. 7. Normalize energy cost

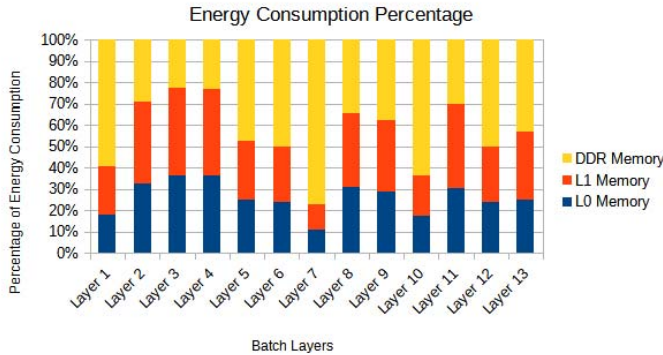


Fig. 8. Energy cost percentage for YOLO v3 model

VI. CONCLUSION

We proposed hardware dataflow for CNN accelerator. The proposed dataflow supports multicore architecture and scalable to be implemented in various platforms. The analysis framework for our dataflow is also provided, this framework can be used to estimate the energy consumption and data reusability of our dataflow in various workloads. We also demonstrate our dataflow in FPGA and test its performance using YOLO v3 algorithm. For 1 core implementation, we managed to achieve 30 GOPS performance, data reusability as high as 61.41 times per data, and we managed to minimize energy consumption from DDR memory access as low as 24%.

REFERENCES

- [1] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet Classification With Deep Convolutional Neural Networks," *Communications of the ACM*, vol. 60, no. 6, pp. 84–90, 2017.
- [2] K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," *arXiv.org*, 19-Dec-2014. [Online]. Available: <https://arxiv.org/abs/1409.1556v4>. [Accessed: 02-Jun-2021].
- [3] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016.
- [4] M. Qasaimeh, K. Denolf, J. Lo, K. Vissers, J. Zambreno, and P. H. Jones, "Comparing Energy Efficiency of CPU, GPU and FPGA Implementations for Vision Kernels," 2019 IEEE International Conference on Embedded Software and Systems (ICSS), 2019.
- [5] Y.-H. Chen, J. Emer, and V. Sze, "Using Dataflow to Optimize Energy Efficiency of Deep Neural Network Accelerators," *IEEE Micro*, vol. 37, no. 3, pp. 12–21, 2017.
- [6] N. P. Jouppi, C. Young, N. Patil, D. Patterson, G. Agrawal, R. Bajwa, S. Bates, S. Bhatia, N. Boden, A. Borchers, R. Boyle, P.-luc Cantin, C. Chao, C. Clark, J. Coriell, M. Daley, M. Dau, J. Dean, B. Gelb, T. V. Ghaemmaghami, R. Gottipati, W. Gulland, R. Hagmann, C. R. Ho, D. Hogberg, J. Hu, R. Hundt, D. Hurt, J. Ibarz, A. Jaffey, A. Jaworski, A. Kaplan, H. Khaitan, D. Killebrew, A. Koch, N. Kumar, S. Lacy, J. Laudon, J. Law, D. Le, C. Leary, Z. Liu, K. Lucke, A. Lundin, G. MacKean, A. Maggiore, M. Mahony, K. Miller, R. Nagarajan, R. Narayanaswami, R. Ni, K. Nix, T. Norrie, M. Omernick, N. Penukonda, A. Phelps, J. Ross, M. Ross, A. Salek, E. Samadiani, C. Severn, G. Sizikov, M. Snellman, J. Souter, D. Steinberg, A. Swing, M. Tan, G. Thorson, B. Tian, H. Toma, E. Tuttle, V. Vasudevan, R. Walter, W. Wang, E. Wilcox, and D. H. Yoon, "In-Datcenter Performance Analysis of a Tensor Processing Unit," *Proceedings of the 44th Annual International Symposium on Computer Architecture*, 2017.
- [7] M. Sankaradas, V. Jakkula, S. Cadambi, S. Chakradhar, I. Durdanovic, E. Cosatto, and H. P. Graf, "A Massively Parallel Coprocessor for Convolutional Neural Networks," 2009 20th IEEE International Conference on Application-specific Systems, Architectures and Processors, 2009.
- [8] L. Cavigelli and L. Benini, "Origami: A 803-GOP/s/W Convolutional Neural Network Accelerator," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 27, no. 11, pp. 2461–2475, 2017.
- [9] B. Moons and M. Verhelst, "A 0.3–2.6 TOPS/W precision-scalable processor for real-time large-scale ConvNets," 2016 IEEE Symposium on VLSI Circuits (VLSI-Circuits), 2016.
- [10] Z. Du, R. Fasthuber, T. Chen, P. Ienne, L. Li, T. Luo, X. Feng, Y. Chen, and O. Temam, "ShiDianNao," *Proceedings of the 42nd Annual International Symposium on Computer Architecture*, 2015.
- [11] A. Parashar, M. Rhu, A. Mukkara, A. Puglielli, R. Venkatesan, B. Khailany, J. Emer, S. W. Keckler, and W. J. Dally, "SCNN," *ACM SIGARCH Computer Architecture News*, vol. 45, no. 2, pp. 27–40, 2017.
- [12] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, "Eyeriss: An Energy-Efficient Reconfigurable Accelerator for Deep Convolutional Neural Networks," *IEEE Journal of Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, 2017.
- [13] Y.-H. Chen, T.-J. Yang, J. S. Emer, and V. Sze, "Eyeriss v2: A Flexible Accelerator for Emerging Deep Neural Networks on Mobile Devices," *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*, vol. 9, no. 2, pp. 292–308, 2019.
- [14] J. Cong and B. Xiao, "Minimizing Computation in Convolutional Neural Networks," *Artificial Neural Networks and Machine Learning – ICANN 2014*, pp. 281–290, 2014.
- [15] M. Dukhan, "The Indirect Convolution Algorithm," *arXiv.org*, 03-Jul-2019. [Online]. Available: <https://arxiv.org/abs/1907.02129>. [Accessed: 03-Jun-2021].
- [16] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, "Efficient Processing of Deep Neural Networks: A Tutorial and Survey," *Proceedings of the IEEE*, vol. 105, no. 12, pp. 2295–2329, 2017.
- [17] J. Redmon and A. Farhadi, "YOLOv3: An Incremental Improvement," *arXiv.org*, 08-Apr-2018. [Online]. Available: <https://arxiv.org/abs/1804.02767>. [Accessed: 07-Jun-2021].